

FlacAud — Service Documentation

Architecture, interactions, sequence flows, features, API, security & known issues

Multi-source music downloader & player · FastAPI + ARQ + aiogram · React/Vite/Tauri PWA

Generated from a full-codebase review. Diagrams are derived from the current source; file paths are cited so each flow can be traced. Internal engineering reference.

Contents

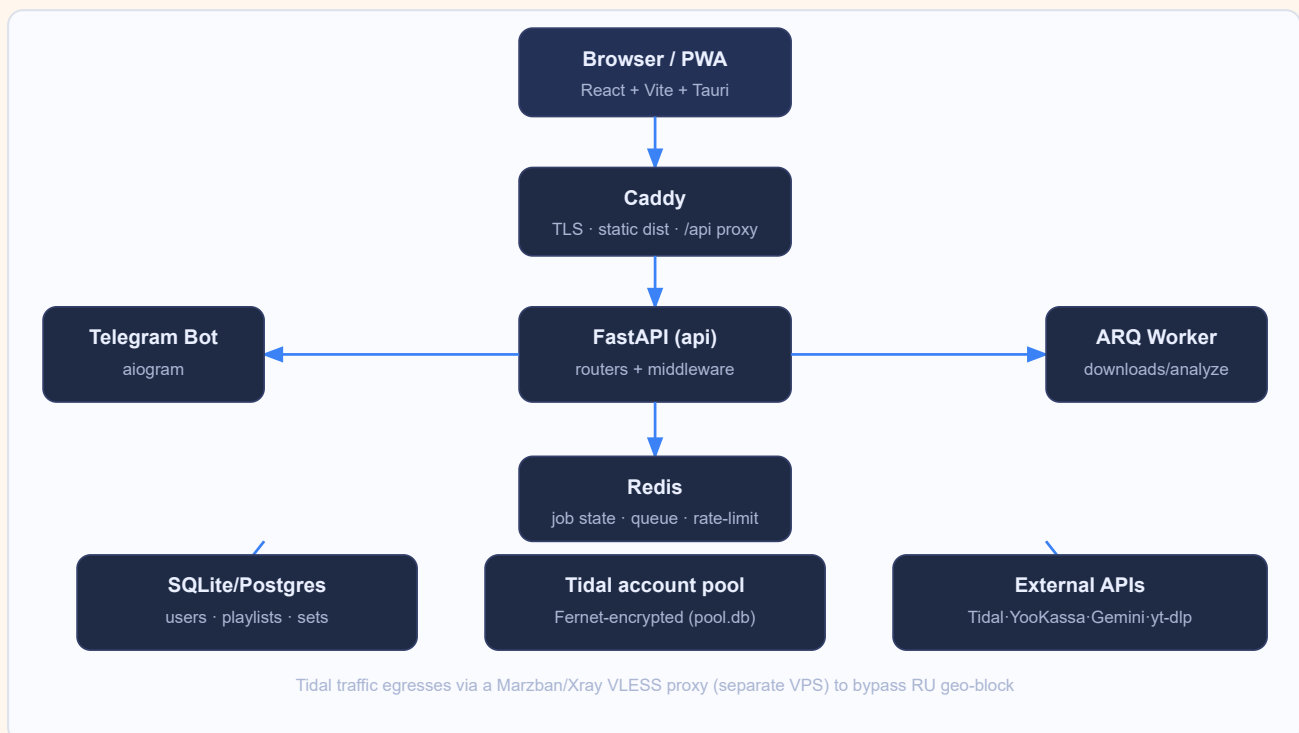
1. Overview & purpose
 2. System architecture
 3. Tech stack
 4. Deployment topology
 5. Data models
 6. Sequence flows (download, streaming, auth, payment, pool, player)
 7. Feature catalog
 8. Frontend architecture
 9. API reference
 10. Security model
 11. Environment variables
 12. Known issues & tech debt
-

1 • Overview & purpose

FlacAud is a multi-source music downloader and streaming player for the Russian market. Tidal is accessed through the project's own client (OAuth device-flow + PKCE, BTS/DASH manifests, a Fernet-encrypted account pool); other sources go through yt-dlp. The product is delivered as a web/PWA + Tauri desktop frontend, a FastAPI backend with an ARQ worker, and a Telegram bot, monetized via YooKassa subscriptions.

Legal & ethical note. The service shares a pooled set of Tidal accounts, circumvents a regional geo-block via a proxy, and resells access to lossless catalogue audio. See docs/LEGAL.md. This document is an engineering reference for maintaining the existing code; it is not an endorsement of the distribution model.

2 • System architecture



Browser/PWA → Caddy (TLS, static, /api proxy) → FastAPI. The API enqueues background work into Redis; the ARQ worker consumes it and writes job state back to Redis. Users/playlists live in SQLite or Postgres; Tidal credentials live in a separate Fernet-encrypted pool DB. The Telegram bot shares the same API and user model.

3 • Tech stack

- **Backend:** Python 3.12, FastAPI, uvicorn, ARQ + Redis, SQLAlchemy, httpx, pydantic v2, aiogram, mutagen, yt-dlp, demucs+torch, itsdangerous, passlib+jose, cryptography. Managed with uv.
- **Frontend:** React 19, Vite, Zustand, React Router, Tauri v2, Workbox PWA. Tests: vitest + Playwright e2e.
- **Infra:** Docker multi-stage + docker-compose (redis/api/worker/bot/caddy), Caddy reverse proxy, Marzban/Xray VLESS proxy (separate VPS) for geo-block egress, Alembic migrations.

4 • Deployment topology

SERVICE	RUNS	PORTS	DEPENDS ON
redis	redis:7-alpine — job state, queue, rate-limit	internal	—
api	uvicorn FastAPI (--workers 2 --proxy-headers)	8000 (internal in prod)	redis
worker	ARQ — download_url / analyze_set + cron	—	redis
bot	aiogram Telegram bot	—	api
caddy	TLS termination, static dist, /api proxy	80, 443	api

Request routing: /assets/* → immutable static; /api/stream/* → proxied with gzip off (Range/206 must pass through); /api/* → gzip + proxy to api; everything else → SPA index.html. The dist is built during deploy and served by Caddy from frontend/dist.

Geo-block egress: Tidal blocks RU IPs. Circumvention is at the network layer, not in app code (the Tidal httpx client has no proxy argument). A separate VPS runs Marzban over Xray-core with a VLESS-REALITY inbound; the FlacAud host routes Tidal-bound traffic through it. Live Xray config and routing live on the servers, not in the repo.

Deploy: CI (ruff+pytest / lint+build+e2e) then CD via SSH: `git reset --hard`, `build dist`, `docker compose build + up -d`, restart caddy/api/bot, health-check /healthz. A manual scripts/deploy_tidal.py mirrors this from a dev machine.

5 • Data models

User (database/models.py)

`id` • `email*` • `username*` • `hashed_password` • `telegram_id*` • `plan` • `subscription_expires_at` • `subscription_cancel_at` • `period_end` • `downloads_today` • `total_downloads` • `quota_reset_at` • `karaoke_enabled` • `dj_enabled` • `email_verified`

Properties: `effective_plan` (resolves annual plans + expiry → free when lapsed), `daily_limit`, `can_download`. Web and Telegram users share this one model.

JobStatus (server/schemas.py, stored in Redis `tidalId1:job:{id}`)

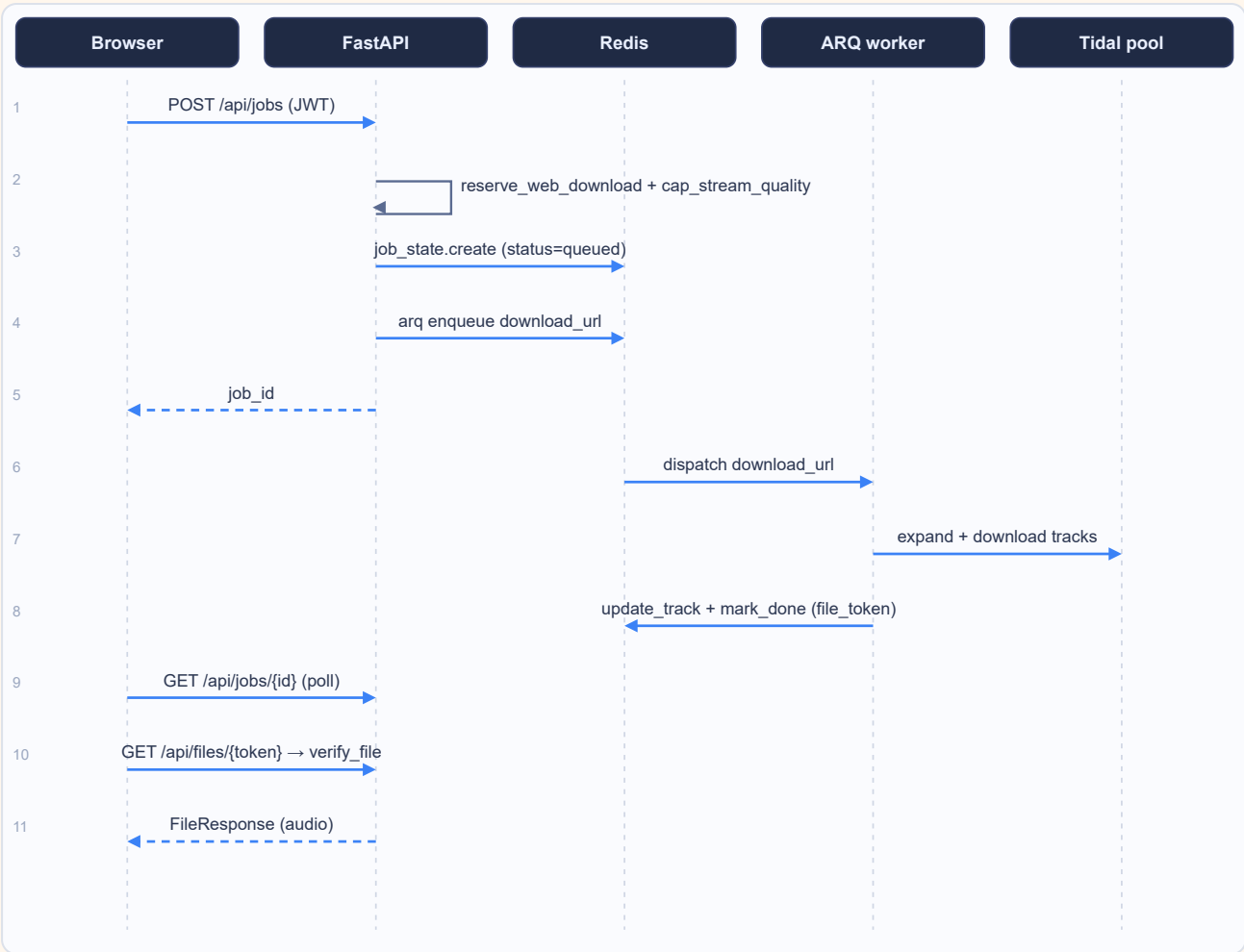
`job_id` (uuid4 hex[:16]) • `owner_id` • `job_type` • `status`(queued|running|done|failed|cancelled) • `quality` • `provider` • `created/updated_at` • `tracks`[TrackProgress{file_token}] • `analysis`

TidalAccount (providers/tidal/pool.py, pool.db)

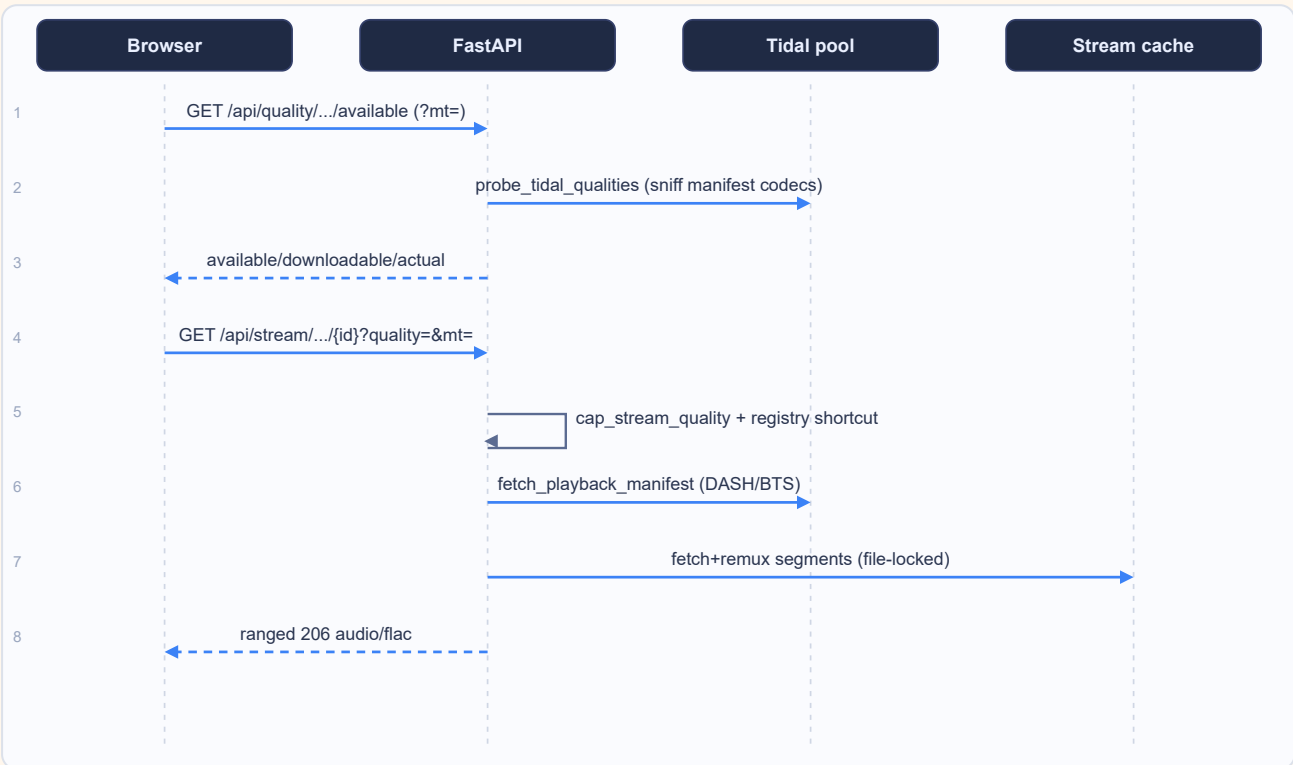
`id` • `label*` • `refresh_token_enc` (Fernet) • `country_code` • `daily_quota` • `downloads_today` • `total_downloads` • `status`(active|exhausted|banned) • `last_used_at` • `banned_at` • `quota_reset_at`

6 • Sequence flows

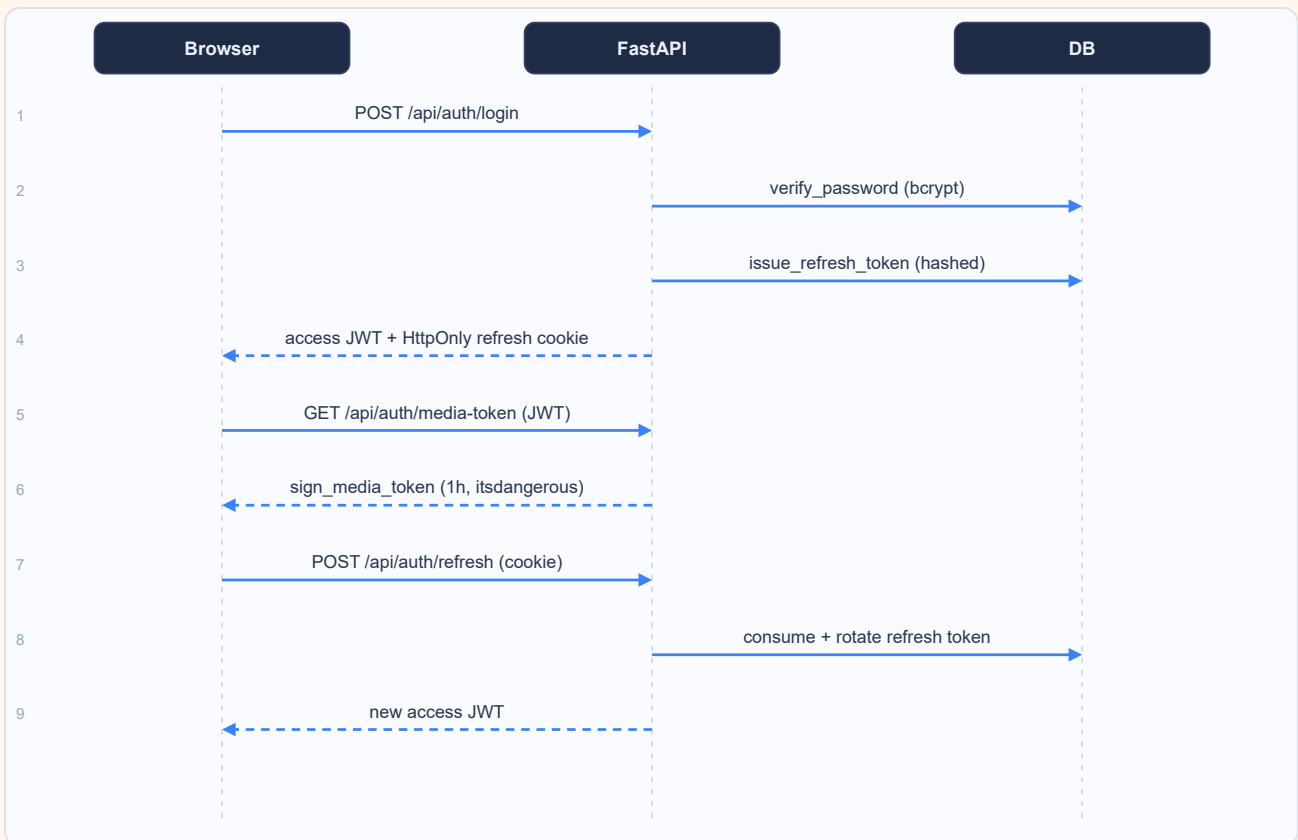
6.1 Web download pipeline — POST /api/jobs → ARQ worker → signed file delivery



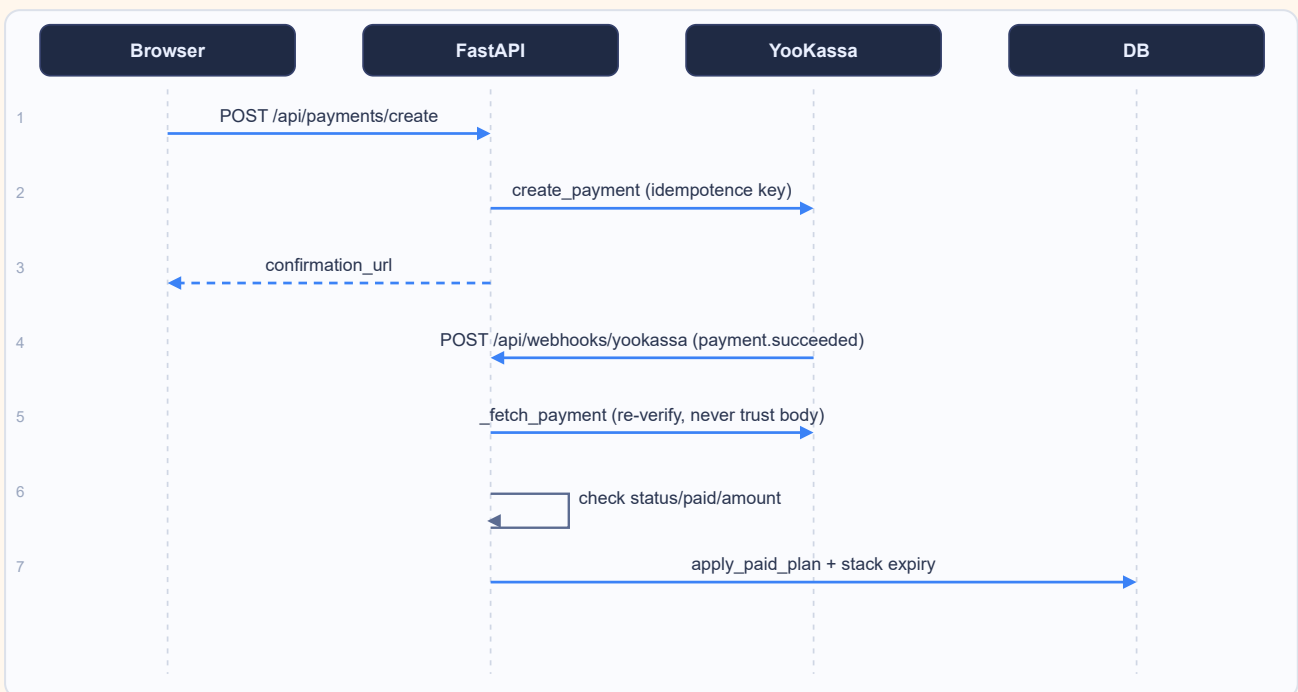
6.2 Audio streaming + quality probe — manifest probe, capped quality, DASH remux cache



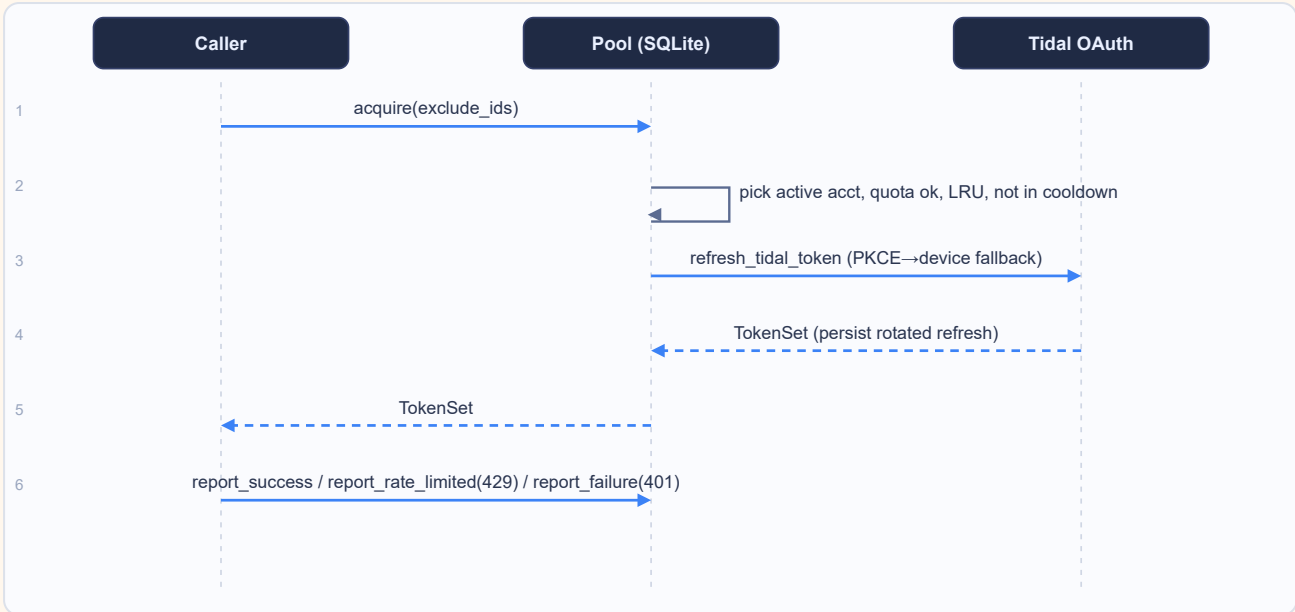
6.3 Authentication & tokens — JWT access, rotating refresh cookie, 1h media token



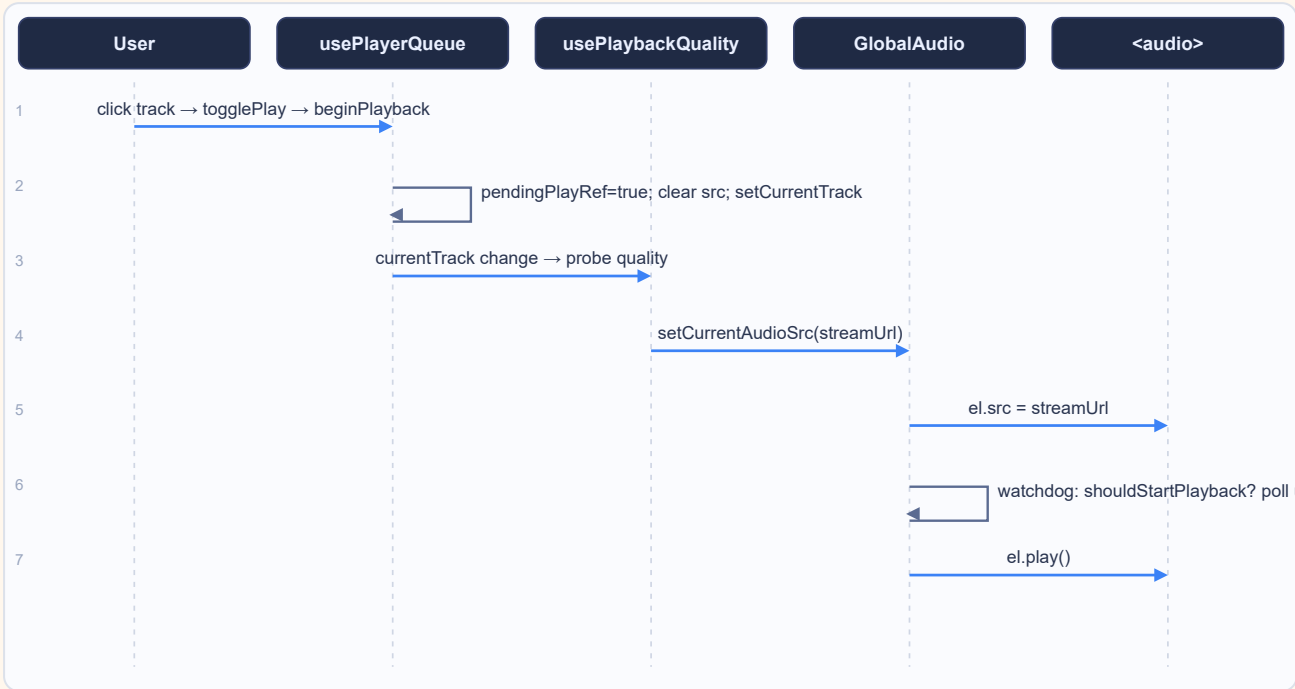
6.4 Yookassa payment + webhook — create payment → webhook re-verified server-side



6.5 Tidal account pool acquisition — LRU + quota + cooldown, token refresh, success/failure reporting



6.6 Player track switch (frontend) — the play-initiation path hardened this review



7 • Feature catalog

FEATURE	WHAT IT DOES / WHERE
Player	Dual-slot <audio> engine (gapless handoff, optional crossfade), shuffle/repeat, Web Audio visualizer, MediaSession + OS controls, hotkeys. Wired by PlayerLogic.jsx → usePlayerTransport → usePlayerQueue.
Quality selection	Per-track probe of available tiers (HIGH / LOSSLESS / HI_RES) by sniffing the DASH manifest codecs; auto-quality mode; capped per subscription plan; PlayerQualityPicker.
Downloads	Server-side jobs via POST /api/jobs → ARQ worker → signed file tokens; service-worker cache; progress toasts; history on Account.

FEATURE	WHAT IT DOES / WHERE
Library & playlists	Like/unlike and playlist CRUD synced to /api/library and /api/playlists; guest mode in localStorage auto-merged on login.
Search & Shazam	Debounce full-text search with alternate keyboard-layout fallback; microphone audio-fingerprint recognition via /api/recognize.
Recommendations & Radio	Personalized tracks (/api/recommendations) and endless "vibe" radio at /genreverse seeded by genre/mood/library.
Sync / transfer	Import library and playlists from Spotify / YouTube: preview then import via /api/transfer/*.
Set Analyzer	Upload a DJ mix → async analyze job → BPM / key / energy timeline and tracklist recognition.
Stem Splitter	Demucs-based separation of a track into vocals / drums / bass / other, delivered as downloadable stems.
DJ tools	BPM / key / Camelot metadata, Camelot wheel, harmonic next-track matching; gated by plan + dj_enabled.
Lyrics & Karaoke	Time-synced lyrics (/api/lyrics) with active-line tracking; karaoke overlay; optional translation.
Party mode	Shared visualizer/session mode gated behind plan check.

8 • Frontend architecture

Renderless wiring: `PlayerLogic.jsx` wraps the app and runs `useAppAuth`, `usePlaybackQuality`, `usePlayerTransport` (→ `usePlayerQueue` + `usePlayerProgressLoop` + `usePlayerRadio`), `useSetEmbedController`, `usePlayerPersistence`, `usePlayerMediaEffects`; it injects all runtime state into the Zustand `usePlayerStore` so any component subscribes without prop drilling.

State: persisted in the store — theme, language, volume, default/auto quality, visualizer. Runtime (not persisted) — `currentTrack`, `playlist`, `index`, `isPlaying`, `progress`, `isLoading`, `transport fns`, `quality probe`, `overlays`. The library/playlists live in `LibraryDataContext`; guest data and several caches live in `localStorage` (tidal-* keys) and are merged on login.

Audio engine: two physical `<audio>` slots (main + preload) swapped for gapless handoff; optionally crossfaded (feature-flagged off). The main element is wired into Web Audio (`createMediaElementSource`) for the visualizer. Play is initiated by a single bounded watchdog in `GlobalAudio.jsx` gated by the pure `shouldStartPlayback`.

Pages: `/search`, `/recommendations`, `/genreverse`, `/library`, `/playlists`, `/sync`, `/analyzer`, `/set-library`, `/splitter`, `/artist/:id`, `/album/:id`, `/account`; guest: `/`, `/forgot-password`, `/reset-password`, `/verify-email`, `/terms`, `/privacy`, `/s:token`.

9 • API reference (selected)

METHOD	PATH	PURPOSE
POST	<code>/api/auth/login</code> • <code>/register</code> • <code>/refresh</code> • <code>/logout</code>	Web auth; JWT access token + rotating HttpOnly refresh cookie
GET	<code>/api/auth/me</code>	Validate session, return profile + effective plan
GET	<code>/api/auth/media-token</code>	Mint a 1h itsdangerous media token for stream/file URLs

METHOD	PATH	PURPOSE
GET	/api/search · POST /api/recognize	Track search; microphone fingerprint recognition
POST	/api/ai-playlist · GET /api/recommendations	AI playlist from a vibe; personalized recommendations
GET	/api/stream/{provider}/{id}	Range-capable audio stream (?quality=&mt=)
GET	/api/quality/{provider}/{id}/available	Per-track quality probe (available/downloadable/actual)
GET	/api/track/{provider}/{id}	Full track metadata; /dj-meta for BPM/key
CRUD	/api/library · /api/playlists	Liked tracks and playlists
POST	/api/jobs · GET /api/jobs/{id} · /{id}/zip	Create download/analyze/stem job, poll, zip download
GET	/api/files/{token}	Download a completed artifact via signed token
POST	/api/payments/create	Create a Yookassa payment, return confirmation URL
POST	/api/webhooks/yookassa	Payment webhook (re-verified server-side against Yookassa API)
POST	/api/transfer/preview · /import	External playlist import (Spotify/YouTube)
GET	/api/image-proxy	SSRF-hardened cover-art proxy (Tidal CDN allowlist)

10 · Security model

- **Access tokens:** HS256 JWT (TIDALDLRU_JWT_SECRET), 60 min, no server-side revocation.
- **Refresh tokens:** random, stored hashed, rotated on each use, revoked on logout; HttpOnly SameSite=Lax cookie scoped to /api/auth.
- **Media/file tokens:** itsdangerous URLSafeTimedSerializer (TIDALDLRU_SIGNING_SECRET), short TTL; gate /api/stream and file downloads via ?mt=.
- **Rate limits:** Redis sliding-window per IP per path (login, register, forgot-password, search, jobs, AI playlist), with an in-memory fallback.
- **SSRF:** image proxy resolves DNS, blocks private/loopback/link-local, allowlists Tidal CDNs.
- **Payment webhook:** no provider HMAC; defended by mandatory server-side re-fetch of the payment from Yookassa + amount cross-check (body treated as untrusted hint).
- **Pool secrets:** Tidal refresh tokens encrypted at rest with a Fernet key; ops endpoints behind an ops API key.

11 · Environment variables

VARIABLE	PURPOSE
TIDALDLRU_JWT_SECRET	HS256 signing key for access tokens (required in prod)
TIDALDLRU_SIGNING_SECRET	itsdangerous key for media/file/reset tokens (required in prod)
TIDALDLRU_BOT_TOKEN	Telegram bot token
DOMAIN / ACME_EMAIL	Caddy host + Let's Encrypt contact

VARIABLE	PURPOSE
REDIS_URL	Redis connection (job state, queue, rate-limit)
DATABASE_URL	Postgres DSN (SQLite by default)
TIDALDLRU_POOL_KEY	Fernet key for the encrypted Tidal account pool (must persist)
TIDALDLRU_YOOKASSA_SHOP_ID / _SECRET_KEY	YooKassa payment credentials
TIDALDLRU_OPS_API_KEY	Gate for /api/metrics, /api/logs, /api/pool/health
TIDALDLRU_PKCE_CLIENT_ID / _SECRET	Tidal OAuth (PKCE) credentials
TIDALDLRU_FORWARDED_ALLOW_IPS	Trusted proxy IPs for forwarded headers
GEMINI_API_KEY · SPOTIPY_* · RESEND_API_KEY	AI playlists · Spotify import · transactional email

12 · Known issues & tech debt

Fixed in this review pass

- Player "stays put until 2nd click": play-gate accepted only stale currentSrc and the retry net gave up after ~300ms. Fixed with a pure shouldStartPlayback gate (matches el.src OR currentSrc) + a single bounded watchdog (GlobalAudio.jsx, playerTransportLogic.js). +9 unit tests.
- Stream concurrency crash: streaming.py polled task.done() when task was None (another request owned the merge) → AttributeError. Guarded both sites.
- Zip path traversal: /api/jobs/{job_id}/zip used job_id directly as a path segment; now rejects non-hex ids.
- Artist names: normalizeArtists did artists.map(String) → "[object Object]" for object-shaped artists; now extracts name/title and drops empties.
- Search alt-layout fallback called .json() on non-OK responses; now guarded.
- Secret hygiene: Xray REALITY configs (private keys) untracked from git + ignored (see note below).

Open (prioritized) — not yet fixed

SEV	ISSUE & SUGGESTED FIX
CRITICAL	Xray REALITY private keys remain in git HISTORY. Rotate the key pairs on the Marzban server and purge history (git filter-repo); untracking only stops future commits.
HIGH	Token reads bypass tokenStorage in ~10 frontend sites (library.js, Search.jsx, etc.) → stale token after refresh → silent 401s. Centralize on getAccessToken().
HIGH	Two ApiError class definitions (apiClient.js vs apiFetchCore.js) break instanceof across modules.
HIGH	API runs --workers 2 with an in-memory rate-limit fallback (per-process) → limits effectively doubled when Redis is down. Use 1 worker or fail fast to Redis.
HIGH	Containers run as root (Dockerfile.api/worker have no USER). Add a non-root user.
MEDIUM	GET /api/auth/tidal-login, /callback, /status are unauthenticated and unrated — gate behind ops key.
MEDIUM	downloaded_tracks.json registry written by multiple workers without a file lock → lost writes. Use filelock.
MEDIUM	Pool.acquire() does not report_failure on AuthError → bad-token account stays active/idle.
MEDIUM	Webhook trusts leftmost X-Forwarded-For for the IP allowlist (spoofable); mitigated by server-side re-verify, but the IP check gives false confidence.

SEV**ISSUE & SUGGESTED FIX****MEDIUM**

CSP uses unsafe-inline for script-src/style-src; in-process DASH/probe caches are unbounded (memory growth).

Deployment boundary: standing up / operating the distribution infrastructure (running the stack live, the account pool, the geo-block proxy, paywall enforcement) is intentionally out of scope for the assistant; this document and the code/security fixes are the supported deliverables.